Scenario- Based SQL INTERVIEW QUESTIONS

DATACEPS

1. FIND DUPLICATE RECORDS IN A TABLE (AMAZON)

Write a query to find rows that appear more than once in a table.

```
SELECT column1, column2, COUNT(*)
FROM your_table
GROUP BY column1, column2
HAVING COUNT(*) > 1;
```

2. GET THE SECOND HIGHEST SALARY (EMPLOYEE TABLE)

Write a query to find the second highest salary from the Employee table.

```
SELECT MAX(salary) AS SecondHighestSalary
FROM Employee
WHERE salary < (SELECT MAX(salary) FROM
Employee);
```

3. FIND EMPLOYEES WITHOUT A DEPARTMENT (UBER)

Write a query to list employees who are not linked to any department.

```
SELECT e.*
FROM Employee e
LEFT JOIN Department d
  ON e.department_id = d.department_id
WHERE d.department_id IS NULL;
```

4. CALCULATE TOTAL REVENUE PER PRODUCT (PAYPAL)

Write a query to calculate the total revenue for each product.

```
SELECT product_id,
       SUM(quantity * price) AS
total_revenue
FROM Sales
GROUP BY product_id;
```

5. GET TOP 3 HIGHEST PAID EMPLOYEES (GOOGLE)

Write a query to find the top three employees with the highest salaries.

```
SELECT *
FROM Employee
ORDER BY salary DESC
LIMIT 3;
```

6. CUSTOMERS WHO PURCHASED BUT NEVER RETURNED PRODUCTS (WALMART)

Write a query to find customers who made purchases but never returned any product.

```
SELECT DISTINCT c.customer_id
FROM Customers c
JOIN Orders o
  ON c.customer_id = o.customer_id
WHERE c.customer_id NOT IN (
    SELECT customer_id FROM Returns
);
```

7. COUNT ORDERS PER CUSTOMER (META)

Write a query to show how many orders each customer has placed.

```
SELECT customer_id,
       COUNT(*) AS order_count
FROM Orders
GROUP BY customer_id;
```

8. FIND EMPLOYEES WHO JOINED IN 2023 (AMAZON)

Write a query to list employees who joined in the year 2023.

```
SELECT *
FROM Employee
WHERE EXTRACT(YEAR FROM hire_date) =
2023;
```

9. CALCULATE AVERAGE ORDER VALUE PER CUSTOMER (MICROSOFT)

Write a query to calculate the average order value for each customer.

```
SELECT customer_id,
       AVG(total_amount) AS
avg_order_value
FROM Orders
GROUP BY customer_id;
```

10. GET THE LATEST ORDER FOR EACH CUSTOMER (UBER)

Write a query to find the most recent order placed by each customer.

```
SELECT customer_id,
       MAX(order_date) AS
latest_order_date
FROM Orders
GROUP BY customer_id;
```

11. FIND PRODUCTS THAT WERE NEVER SOLD

Write a query to list products that have never been sold.

```
SELECT p.product_id
FROM Products p
LEFT JOIN Sales s
  ON p.product_id = s.product_id
WHERE s.product_id IS NULL;
```

12. IDENTIFY THE MOST SELLING PRODUCT (ADOBE / WALMART)

Write a query to find the product that sold the most quantity.

```
SELECT product_id,
       SUM(quantity) AS total_qty
FROM Sales
GROUP BY product_id
ORDER BY total_qty DESC
LIMIT 1;
```

13. GET TOTAL REVENUE AND ORDER COUNT PER REGION (META)

Write a query to calculate total revenue and total number of orders for each region.

```
SELECT region,
       SUM(total_amount) AS
total_revenue,
       COUNT(*) AS order_count
FROM Orders
GROUP BY region;
```

14. COUNT CUSTOMERS WITH MORE THAN 5 ORDERS (AMAZON)

Write a query to count how many customers have placed more than five orders.

```
SELECT COUNT(*) AS customer_count
FROM (
    SELECT customer_id
    FROM Orders
    GROUP BY customer_id
    HAVING COUNT(*) > 5
) AS subquery;
```

15. FIND CUSTOMERS WITH ORDERS ABOVE AVERAGE VALUE (PAYPAL)

Write a query to list customers whose order value is higher than the average order value.

```
SELECT DISTINCT customer_id
FROM Orders
WHERE total_amount > (
    SELECT AVG(total_amount) FROM Orders
);
```

16. FIND EMPLOYEES HIRED ON WEEKENDS (GOOGLE)

Write a query to list employees who were hired on a Saturday or Sunday.

```
SELECT *
FROM Employee
WHERE EXTRACT(DOW FROM hire_date) IN (0,
6);
```

17. FIND EMPLOYEES WITH SALARY BETWEEN 50,000 AND 100,000 (MICROSOFT)

Write a query to list employees whose salary falls between 50,000 and 100,000.

```
SELECT *
FROM Employee
WHERE salary BETWEEN 50000 AND 100000;
```

18. GET MONTHLY SALES REVENUE AND ORDER COUNT (GOOGLE)

Write a query to calculate total revenue and number of orders for each month.

```
SELECT TO_CHAR(order_date, 'YYYY-MM') AS
month,
       SUM(total_amount) AS
total_revenue,
       COUNT(order_id) AS order_count
FROM Orders
GROUP BY TO_CHAR(order_date, 'YYYY-MM');
```

19. RANK EMPLOYEES BY SALARY WITHIN EACH DEPARTMENT (AMAZON)

Write a query to rank employees based on salary within their department.

```
SELECT employee_id,
       department_id,
       salary,
       RANK() OVER (
           PARTITION BY department_id
           ORDER BY salary DESC
       ) AS salary_rk
FROM Employee;
```

20. FIND CUSTOMERS WHO PLACED ORDERS EVERY MONTH IN 2023 (META)

Write a query to find customers who placed at least one order in every month of 2023.

```
SELECT customer_id
FROM Orders
WHERE EXTRACT(YEAR FROM order_date) =
2023
GROUP BY customer_id
HAVING COUNT(DISTINCT TO_CHAR(order_date,
'YYYY-MM')) = 12;
```

21. CALCULATE 3-DAY MOVING AVERAGE OF SALES (MICROSOFT)

Write a query to calculate the moving average of sales over the last three days.

```
SELECT order_date,
       AVG(total_amount) OVER (
           ORDER BY order_date
           ROWS BETWEEN 2 PRECEDING AND
CURRENT ROW
       ) AS moving_avg
FROM Orders;
```

22. FIND FIRST AND LAST ORDER DATE FOR EACH CUSTOMER (UBER)

Write a query to find the first and last order date for each customer.

```
SELECT customer_id,
       MIN(order_date) AS first_order,
       MAX(order_date) AS last_order
FROM Orders
GROUP BY customer_id;
```

23. SHOW PRODUCT SALES DISTRIBUTION (PERCENT OF TOTAL REVENUE) (PAYPAL)

Write a query to calculate each product's share of total revenue as a percentage.

```
WITH TotalRevenue AS (
    SELECT SUM(quantity * price) AS total
    FROM Sales
)
SELECT s.product_id,
       SUM(s.quantity * s.price) AS
revenue,
       SUM(s.quantity * s.price) * 100 /
t.total AS revenue_pct
FROM Sales s
CROSS JOIN TotalRevenue t
GROUP BY s.product_id, t.total;
```

24. FIND CUSTOMERS WITH CONSECUTIVE PURCHASES (2 DAYS) (WALMART)

Write a query to find customers who made purchases on two consecutive days.

```
WITH cte AS (
    SELECT customer_id,
           order_date,
           LAG(order_date) OVER (
               PARTITION BY customer_id
               ORDER BY order_date
           ) AS prev_order_date
    FROM Orders
)
SELECT customer_id,
       order_date,
       prev_order_date
FROM cte
WHERE order_date - prev_order_date =
INTERVAL '1 DAY';
```

25. FIND CHURNED CUSTOMERS (NO ORDERS IN LAST 6 MONTHS) (AMAZON)

Write a query to find customers who have not placed any order in the last six months.

```sql
SELECT customer_id
FROM Orders
GROUP BY customer_id
HAVING MAX(order_date) < (NOW() -
INTERVAL '6 months');
```

26. CALCULATE CUMULATIVE REVENUE BY DAY (ADOBE)

Write a query to calculate running total revenue by date.

```sql
SELECT order_date,
       SUM(total_amount) OVER (
           ORDER BY order_date
       ) AS cumulative_revenue
FROM Orders;
```

27. FIND DEPARTMENTS WITH HIGHEST AVERAGE SALARY (GOOGLE)

Write a query to rank departments based on their average salary.

```
SELECT department_id,
       AVG(salary) AS avg_salary
FROM Employee
GROUP BY department_id
ORDER BY avg_salary DESC;
```

28. FIND CUSTOMERS WHO ORDERED MORE THAN AVERAGE NUMBER OF ORDERS (META)

Write a query to list customers who placed more orders than the average customer.

```
WITH customer_orders AS (
    SELECT customer_id,
           COUNT(*) AS order_count
    FROM Orders
    GROUP BY customer_id
)
SELECT *
FROM customer_orders
WHERE order_count > (
    SELECT AVG(order_count) FROM
customer_orders
);
```

29. CALCULATE REVENUE FROM NEW CUSTOMERS (FIRST-TIME ORDERS) (MICROSOFT)

Write a query to calculate revenue generated from customers’ first orders.

```
WITH first_orders AS (
    SELECT customer_id,
           MIN(order_date) AS
first_order_date
    FROM Orders
    GROUP BY customer_id
)
SELECT SUM(o.total_amount) AS new_revenue
FROM Orders o
JOIN first_orders f
  ON o.customer_id = f.customer_id
WHERE o.order_date = f.first_order_date;
```

30. CALCULATE PERCENTAGE OF EMPLOYEES IN EACH DEPARTMENT (UBER)

Write a query to calculate what percentage of total employees belong to each department.

```
SELECT department_id,
       COUNT(*) AS emp_count,
       COUNT(*) * 100.0 /
       (SELECT COUNT(*) FROM Employee) AS
pct
FROM Employee
GROUP BY department_id;
```

31. FIND MAXIMUM SALARY DIFFERENCE WITHIN EACH DEPARTMENT (PAYPAL)

Write a query to calculate the salary gap (difference between highest and lowest salary) in each department.

```
SELECT department_id,
       MAX(salary) - MIN(salary) AS
salary_diff
FROM Employee
GROUP BY department_id;
```

32. FIND PRODUCTS CONTRIBUTING TO 80% OF REVENUE (PARETO PRINCIPLE) (WALMART)

Write a query to identify products that together contribute to 80% of total revenue.

```
WITH sales_cte AS (
    SELECT product_id,
           SUM(quantity * price) AS
revenue
    FROM Sales
    GROUP BY product_id
),
total_revenue AS (
    SELECT SUM(revenue) AS total FROM
sales_cte
)
SELECT product_id,
       revenue,
       cumulative_revenue
FROM (
    SELECT s.product_id,
           s.revenue,
           SUM(s.revenue) OVER (ORDER BY
s.revenue DESC)
             AS cumulative_revenue,
           t.total
    FROM sales_cte s
    CROSS JOIN total_revenue t
) sub
WHERE cumulative_revenue <= total * 0.8;
```

33. SHOW LAST PURCHASE FOR EACH CUSTOMER WITH ORDER AMOUNT (GOOGLE)

Write a query to find the most recent order for each customer along with the order amount.

```
WITH ranked_orders AS (
    SELECT customer_id,
           order_id,
           total_amount,
           ROW_NUMBER() OVER (
               PARTITION BY customer_id
               ORDER BY order_date DESC
           ) AS rn
    FROM Orders
)
SELECT customer_id,
       order_id,
       total_amount
FROM ranked_orders
WHERE rn = 1;
```

34. CALCULATE AVERAGE TIME BETWEEN PURCHASES FOR EACH CUSTOMER (META)

Write a query to calculate the average number of days between purchases for each customer.

```
WITH cte AS (
    SELECT customer_id,
           order_date,
           LAG(order_date) OVER (
               PARTITION BY customer_id
               ORDER BY order_date
           ) AS prev_date
    FROM Orders
)
SELECT customer_id,
       AVG(DATE_DIFF(DAY, prev_date,
order_date)) AS avg_gap_days
FROM cte
WHERE prev_date IS NOT NULL
GROUP BY customer_id;
```

35. CALCULATE YEAR-OVER-YEAR REVENUE GROWTH (MICROSOFT)

Write a query to calculate revenue growth compared to the previous year.

```
WITH yearly AS (
    SELECT EXTRACT(YEAR FROM order_date)
AS year,
           SUM(total_amount) AS revenue
    FROM Orders
    GROUP BY EXTRACT(YEAR FROM
order_date)
)
SELECT year,
       revenue,
       revenue - LAG(revenue) OVER (ORDER
BY year) AS yoy_growth
FROM yearly;
```

36. FIND ORDERS ABOVE CUSTOMER'S 90TH PERCENTILE PURCHASE (AMAZON)

Write a query to find orders where the purchase amount is higher than the customer's historical 90th percentile.

```
WITH ranked_orders AS (
    SELECT customer_id,
           order_id,
           total_amount,
           NTILE(10) OVER (
               PARTITION BY customer_id
               ORDER BY total_amount
           ) AS decile
    FROM Orders
)
SELECT customer_id,
       order_id,
       total_amount
FROM ranked_orders
WHERE decile = 10;
```

37. FIND LONGEST GAP BETWEEN ORDERS FOR EACH CUSTOMER (META)

Write a query to calculate the longest time gap between orders for each customer.

```
WITH cte AS (
    SELECT customer_id,
           order_date,
           LAG(order_date) OVER (
               PARTITION BY customer_id
               ORDER BY order_date
           ) AS prev_order_date
    FROM Orders
)
SELECT customer_id,
       MAX(DATE_DIFF(DAY,
prev_order_date, order_date))
         AS max_gap_days
FROM cte
WHERE prev_order_date IS NOT NULL
GROUP BY customer_id;
```

38. IDENTIFY CUSTOMERS WITH REVENUE BELOW 10TH PERCENTILE (GOOGLE)

Write a query to find customers whose total revenue falls below the 10th percentile.

```
WITH cte AS (
    SELECT customer_id,
           SUM(total_amount) AS
total_revenue
    FROM Orders
    GROUP BY customer_id
)
SELECT customer_id,
       total_revenue
FROM cte
WHERE total_revenue < (
    SELECT PERCENTILE_CONT(0.1)
           WITHIN GROUP (ORDER BY
total_revenue)
    FROM cte
);
```